

FreeRTOS消息队列

文章目录

[一. 队列简介](#)

[1、数据存储](#)

[2、多任务访问](#)

[3、出队阻塞](#)

[4、入队阻塞](#)

[5、队列操作过程图示](#)

[二、消息队列常用函数讲解](#)

[1. 消息队列创建函数 xQueueCreate\(\)](#)

[2. 消息队列静态创建函数 xQueueCreateStatic\(\)](#)

[3. 读队列 xQueueReceive\(\)](#)

[4. 写队列 xQueueSend \(\)](#)

[5. 消息队列删除函数 vQueueDelete\(\)](#)

[6. 复位 xQueueReset \(\)](#)

[7. 查询](#)

[三、消息队列使用注意事项](#)

[四、消息队列应用实例](#)

[五、实验现象](#)

一. 队列简介

队列是为了任务与任务、任务与中断之间的通信而准备的，可以在任务与任务、任务与中断之间传递消息，队列中可以存储有限的、大小固定的数据项目。任务与任务、任务与中断之间要交流的数据保存在队列中，叫做队列项目。队列所能保存的最大数据项目数量叫做队列的长度，创建队列的时候会指定数据项目的大小和队列的长度。由于队列用来传递消息的，所以也称为消息队列。FreeRTOS中的信号量的也是依据队列实现的！所以有必要深入的了解FreeRTOS的队列。

1、数据存储

通常队列采用先进先出(FIFO)的存储缓冲机制，也就是往队列发送数据的时候(也叫入队)永远都是发送到队列的尾部，而从队列提取数据的时候(也叫出队)是从队列的头部提取的。但是也可以使用LIFO的存储缓冲机制，也就是后进先出，FreeRTOS中的队列也提供了LIFO的存储缓冲机制。数据发送到队列中会导致数据拷贝，也就是将要发送的数据拷贝到队列中，这就意味着在队列中存储的是数据的原始值，而不是原数据的引用(即只传递数据的指

针)，这个也叫做值传递。学过 UCOS 的同学应该知道，UCOS 的消息队列采用的是引用传递，传递的是消息指针。采用引用传递的话消息内容就必须一直保持可见性，也就是消息内容必须有效，那么局部变量这种可能会随时被删掉的东西就不能用来传递消息，但是采用引用传递会节省时间啊！因为不用进行数据拷贝。采用值传递的话虽然会导致数据拷贝，会浪费一点时间，但是一旦将消息发送到队列中原 始的数据缓冲区就可以删除掉或者覆写，这样的话这些缓冲区就可以被重复的使用。FreeRTOS 中使用队列传递消息的话虽然使用的是数据拷贝，但是也可以使用引用来传递消息啊，我直接 往队列中发送指向这个消息的地址指针不就可以了！这样当我要发送的消息数据太大的时候就可以直接发送消息缓冲区的地址指针，比如在网络应用环境中，网络的数据量往往都很大的，采用数据拷贝的话就不现实。

2、多任务访问

队列不是属于某个特别指定的任务的，任何任务都可以向队列中发送消息，或者从队列中 提取消息。

3、出队阻塞

当任务尝试从一个队列中读取消息的时候可以指定一个阻塞时间，这个阻塞时间就是当任 务从队列中读取消息无效的时候任务阻塞的时间。出队就是从队列中读取消息，出队阻塞是 针对从队列中读取消息的任务而言的。比如任务 A 用于处理串口接收到的数据，串口接收到数 据以后就会放到队列 Q 中，任务 A 从队列 Q 中读取数据。但是如果此时队列 Q 是空的，说明 还没有数据，任务 A 这时候来读取的话肯定是获取不到任何东西，那该怎么办呢？任务 A 现在 有三种选择，一：二话不说扭头就走，二：要不我在等等吧，等一会看看，说不定一会就有数 据了，三：死等，死也要等到你有数据！选哪一个就是由这个阻塞时间决定的，这个阻塞时间 单位是时钟节拍数。阻塞时间为 0 的话就是不阻塞，没有数据的话就马上返回任务继续执行接 下来的代码，对应第一种选择。如果阻塞时间为 0~ portMAX_DELAY，当任务没有从队列中获 取到消息的话就进入阻塞态，阻塞时间指定了任务进入阻塞态的时间，当阻塞时间到了以后还 没有接收到数据的话就退出阻塞态，返回任务接着运行下面的代码，如果在 阻塞时间内接收到了数据就立即返回，执行任务中下面的代码，这种情况对应第二种选择。当阻塞时间设置为 portMAX_DELAY 的话，任务就会一直进入阻塞态等待，直到接收到数据为止！这个就是第三种选择。

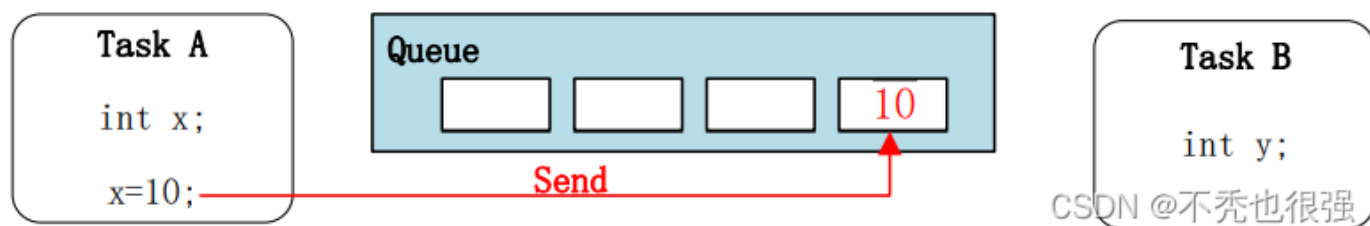
4、入队阻塞

入队说的是向队列中发送消息，将消息加入到队列中。和出队阻塞一样，当一个任务向队 列发送消息的话也可以设置阻塞时间。比如任务 B 向消息队列 Q 发送消息，但是此时队列 Q 是 满的，那肯定是发送失败的。此时任务 B 就会遇到和上面任务 A 一样的问题，这两种情况的处 理过程是类似的，只不过一个是向队列 Q 发送消息，一个是从队列 Q 读取消息而已。

5、队列操作过程图示

下面几幅图简单的演示了一下队列的入队和出队过程。

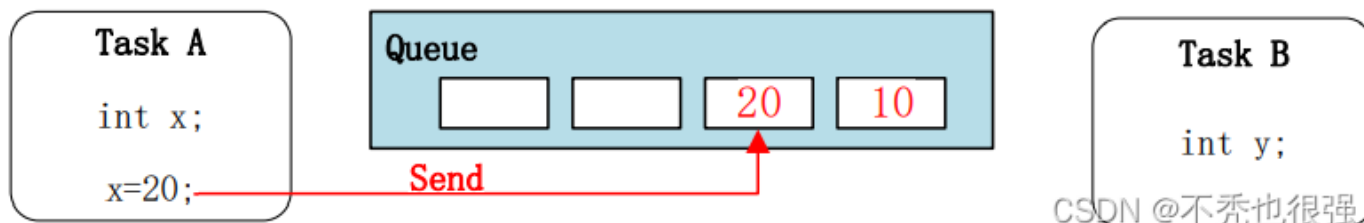
● 向队列发送第一个消息



向队列发送第一个信息

图中任务 A 的变量 x 值为 10，将这个值发送到消息队列中。此时队列剩余长度就是 3 了。前面说了向队列中发送消息是采用拷贝的方式，所以一旦消息发送完成变量 x 就可以再 次被使用，赋其他的值。

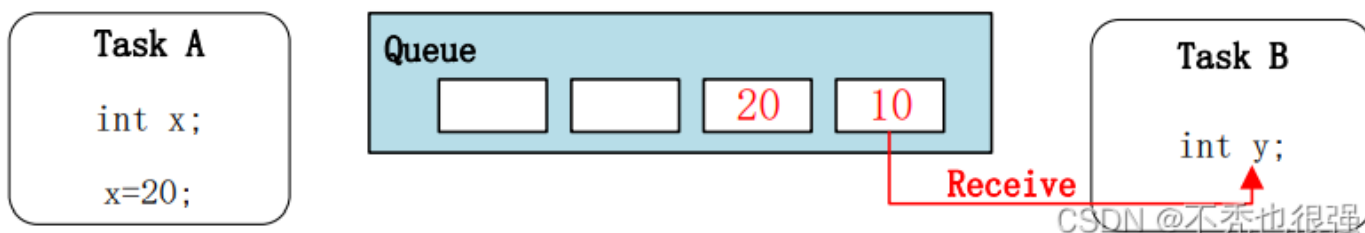
● 向队列发送第二个消息



向队列发送第一个信息

图中任务 A 又向队列发送了一个消息，即新的 x 的值，这里是 20。此时队列剩余长度为 2。

● 从队列中读取消息



图中任务 B 从队列中读取消息，并将读取到的消息值赋值给 y，这样 y 就等于 10 了。任务 B 从队列中读取消息完成以后可以选择清除掉这个消息或者不清除。当选择清除这个消息的话其他任务或中断就不能获取这个消息了，而且队列剩余大小就会加一，变成 3。如果不清除的话其他任务或中断也可以获取这个消息，而队列剩余大小依旧是 2。

二、消息队列常用函数讲解

使用队列模块的典型流程如下：

创建消息队列。

写队列操作。

读队列操作。

删除队列。

1. 消息队列创建函数 xQueueCreate()

xQueueCreate()用于创建一个新的队列并返回可用于访问这个队列的队列句柄。队列句柄其实就是一个指向队列数据结构类型的指针。队列就是一个数据结构，用于任务间的数据的传递。每创建一个新的队列都需要为其分配 RAM，一部分用于存储队列的状态，剩下的作为队列消息的存储区域。使用 xQueueCreate()创建队列时，使用的是动态内存分配，所以要想使用该函数必须在 FreeRTOSConfig.h 中把 configSUPPORT_DYNAMIC_ALLOCATION 定义为 1 来使能，这是个用于使能动态内存分配的宏，通常情况下，在 FreeRTOS 中，凡是创建任务，队列，信号量和互斥量等内核对象都需要使用动态内存分配，所以这个宏默认在 FreeRTOS.h 头文件中已经使能（即定义为 1）。如果想使用静态内存，则可以使用 xQueueCreateStatic() 函数来创建一个队列。使用静态创建消息队列函数创建队列时需要的形参更多，需要的内存由编译的时候预先分配好，一般很少使用这种方法。

xQueueCreate()函数说明


```

6.   Test_Queue = xQueueCreate((UBaseType_t ) QUEUE_LEN, /* 消息队列的长度 */
7.                               (UBaseType_t ) QUEUE_SIZE); /* 消息的大小 */
8.   if(NULL != Test_Queue)
9.       printf("创建Test_Queue消息队列成功!\r\n");
10.  taskEXIT_CRITICAL(); //退出临界区

```



2 消息队列静态创建函数 xQueueCreateStatic()

xQueueCreateStatic()用于创建一个新的队列并返回可用于访问这个队列的队列句柄。队列句柄其实就是一个指向队列数据结构类型的指针。队列就是一个数据结构，用于任务间的数据的传递。每创建一个新的队列都需要为其分配RAM，一部分用于存储队列的状态，剩下的作为队列的存储区。使用xQueueCreateStatic()创建队列时，使用的是静态内存分配，所以要想使用该函数必须在FreeRTOSConfig.h中把configSUPPORT_STATIC_ALLOCATION定义为1来使能。这是个用于使能静态内存分配的宏，需要的内存存在程序编译的时候分配好，由用户自己定义，其实创建过程与xQueueCreate()都是差不多的。

xQueueCreateStatic()函数说明

函数原型	QueueHandle_t xQueueCreateStatic(UBaseType_t uxQueueLength, UBaseType_t uxItemSize, uint8_t *pucQueueStorageBuffer, StaticQueue_t *pxQueueBuffer);	
功能	用于创建一个新的队列。	
参数	uxQueueLength	队列能够存储的最大单元数目，即队列深度。
	uxItemSize	队列中数据单元的长度，以字节为单位。
	pucQueueStorageBuffer	指针，指向一个 uint8_t 类型的数组，数组的大小至少有 uxQueueLength* uxItemSize 个字节。当 uxItemSize 为 0 时，pucQueueStorageBuffer 可以为 NULL。
	pxQueueBuffer	指针，指向 StaticQueue_t 类型的变量，该变量用于存储队列的数据结构。
返回值	如果创建成功则返回一个队列句柄，用于访问创建的队列。如果创建不成功则返回 NULL，可能原因是创建队列需要的 RAM 无法分配成功。	
CSDN @不秃也很强		

xQueueCreateStatic()函数使用实例

```

1.  /* 创建一个可以最多可以存储 10 个 64 位变量的队列 */
2.  #define QUEUE_LENGTH 10
3.
4.  #define ITEM_SIZE sizeof( uint64_t )
5.
6.  /* 该变量用于存储队列的数据结构 */
7.
8.  static StaticQueue_t xStaticQueue;
9.
10. /* 该数组作为队列的存储区域，大小至少有 uxQueueLength * uxItemSize 个字节 */
11.
12. uint8_t ucQueueStorageArea[ QUEUE_LENGTH * ITEM_SIZE ];
13.
14. void vATask( void *pvParameters )
15. {
16.   QueueHandle_t xQueue;
17.
18.   /* 创建一个队列 */
19.   xQueue = xQueueCreateStatic( QUEUE_LENGTH, /* 队列深度 */

```



```
20. ITEM_SIZE, /* 队列数据单元的单位 */
21. ucQueueStorageArea, /* 队列的存储区域 */
22. &xStaticQueue ); /* 队列的数据结构 */
23. /* 剩下的其他代码 */
24. }
```



3.读队列 xQueueReceive()

使用xQueueReceive()函数读队列，读到一个数据后，队列中该数据会被移除。这个函数有两个版本：在任务中使用、在ISR中使用。函数原型如下：

```
1. BaseType_t xQueueReceive( QueueHandle_t xQueue,
2.                           void * const pvBuffer,
3.                           TickType_t xTicksToWait );
4.
5. BaseType_t xQueueReceiveFromISR(
6.                               QueueHandle_t xQueue,
7.                               void *pvBuffer,
8.                               BaseType_t *pxTaskWoken
9.                               );
```

xQueueReceive()函数说明

函数原型	BaseType_t xQueueReceive(QueueHandle_t xQueue, void *pvBuffer, TickType_t xTicksToWait);	
功能	用于从一个队列中接收消息，并把接收的消息从队列中删除。	
参数	xQueue	队列句柄。
	pvBuffer	指针，指向接收到要保存的数据。
	xTicksToWait	队列空时，阻塞超时的最大时间。如果该参数设置为 0，函数立刻返回。超时时间的单位为系统节拍周期，常量 portTICK_PERIOD_MS 用于辅助计算真实的时间，单位为 ms。如果 INCLUDE_vTaskSuspend 设置成 1，并且指定延时为 portMAX_DELAY 将导致任务无限阻塞（没有超时）。
返回值	队列项接收成功返回 pdTRUE，否则返回 pdFALSE。	

CSDN @不秃也很强

xQueueReceive()函数使用实例

```
1. static void Receive_Task(void* parameter)
2. {
3.     BaseType_t xReturn = pdTRUE; /* 定义一个创建信息返回值，默认为pdTRUE */
4.     uint32_t r_queue; /* 定义一个接收消息的变量 */
5.     while (1)
6.     {
7.         xReturn = xQueueReceive( Test_Queue, /* 消息队列的句柄 */
8.                                  &r_queue, /* 发送的消息内容 */
9.                                  portMAX_DELAY); /* 等待时间 一直等 */
10.        if(pdTRUE == xReturn)
11.            printf("本次接收到的数据是%d\n\n", r_queue);
12.        else
13.            printf("数据接收出错, 错误代码0x%lx\n", xReturn);
14.    }
15. }
```



4.写队列 xQueueSend ()

可以把数据写到队列头部，也可以写到尾部，这些函数有两个版本：在任务中使用、在ISR中使用。函数原型如下：

```
1. /* 等同于xQueueSendToBack
2.  * 往队列尾部写入数据，如果没有空间，阻塞时间为xTicksToWait
3.  */
4. BaseType_t xQueueSend(
5.                     QueueHandle_t    xQueue,
6.                     const void *pvItemToQueue,
7.                     TickType_t       xTicksToWait
8.                     );
9.
10. /*
11.  * 往队列尾部写入数据，如果没有空间，阻塞时间为xTicksToWait
12.  */
13. BaseType_t xQueueSendToBack(
14.                     QueueHandle_t    xQueue,
15.                     const void *pvItemToQueue,
16.                     TickType_t       xTicksToWait
17.                     );
18.
19.
20. /*
21.  * 往队列尾部写入数据，此函数可以在中断函数中使用，不可阻塞
22.  */
23. BaseType_t xQueueSendToBackFromISR(
24.                     QueueHandle_t xQueue,
25.                     const void *pvItemToQueue,
26.                     BaseType_t *pxHigherPriorityTaskWoken
27.                     );
28.
29. /*
30.  * 往队列头部写入数据，如果没有空间，阻塞时间为xTicksToWait
31.  */
32. BaseType_t xQueueSendToFront(
33.                     QueueHandle_t    xQueue,
34.                     const void *pvItemToQueue,
35.                     TickType_t       xTicksToWait
36.                     );
37.
38. /*
39.  * 往队列头部写入数据，此函数可以在中断函数中使用，不可阻塞
40.  */
41. BaseType_t xQueueSendToFrontFromISR(
42.                     QueueHandle_t xQueue,
43.                     const void *pvItemToQueue,
44.                     BaseType_t *pxHigherPriorityTaskWoken
45.                     );
```



xQueueSend()函数说明

函数原型	BaseType_t xQueueSend(QueueHandle_t xQueue, const void * pvItemToQueue, TickType_t xTicksToWait);	
功能	用于向队列尾部发送一个队列消息。	
参数	xQueue	队列句柄。
	pvItemToQueue	指针，指向要发送到队列尾部的队列消息。
	xTicksToWait	队列满时，等待队列空闲的最大超时时间。如果队列满并且 xTicksToWait 被设置成 0，函数立刻返回。超时时间的单位为系统节拍周期，常量 portTICK_PERIOD_MS 用于辅助计算真实的时间，单位为 ms。如果 INCLUDE_vTaskSuspend 设置成 1，并且指定延时为 portMAX_DELAY 将导致任务挂起（没有超时）。
返回值	消息发送成功成功返回 pdTRUE，否则返回 errQUEUE_FULL。	

CSDN @不秃也很强

xQueueSend()函数使用实例

```
1. static void Send_Task(void* parameter)
2. {
3.     BaseType_t xReturn = pdPASS; /* 定义一个创建信息返回值，默认为pdPASS */
4.     uint32_t send_data1 = 1;
5.     uint32_t send_data2 = 2;
6.     while (1)
7.     {
8.         if( Key_Scan(KEY1_GPIO_PORT,KEY1_GPIO_PIN) == KEY_ON )
9.             /* K1 被按下 */
10.            printf("发送消息send_data1! \n");
11.            xReturn = xQueueSend( Test_Queue, /* 消息队列的句柄 */
12.                                &send_data1, /* 发送的消息内容 */
13.                                0 );          /* 等待时间 0 */
14.            if(pdPASS == xReturn)
15.                printf("消息send_data1发送成功!\n\n");
16.        }
17.        if( Key_Scan(KEY2_GPIO_PORT,KEY2_GPIO_PIN) == KEY_ON )
18.            /* K2 被按下 */
19.            printf("发送消息send_data2! \n");
20.            xReturn = xQueueSend( Test_Queue, /* 消息队列的句柄 */
21.                                &send_data2, /* 发送的消息内容 */
22.                                0 );          /* 等待时间 0 */
23.            if(pdPASS == xReturn)
24.                printf("消息send_data2发送成功!\n\n");
25.        }
26.        vTaskDelay(20); /* 延时20个tick */
27.    }
28. }
```



5.消息队列删除函数 vQueueDelete()

队列删除函数是根据消息队列句柄直接删除的，删除之后这个消息队列的所有信息都会被系统回收清空，而且不能再次使用这个消息队列了，但是需要注意的是，如果某个消息队列没有被创建，那也是无法被删除的，动脑子想想都知道，没创建的东西就不存在，怎么可能被删除。xQueue是vQueueDelete()函数的形参，是消息队列句柄，表示的是要删除哪个想队列，消息队列删除函数vQueueDelete()的使用也是很简单的，只需传入要删除的消息队列的句柄即可，

调用函数时，系统将删除这个消息队列。需要注意的是调用删除消息队列函数前，系统应存在 xQueueCreate() 或 xQueueCreateStatic()函数创建的消息队列。此外 vQueueDelete()也可用于删除信号量。如果删除消息队列时，有任务正在等待消息，则不应该进行删除操作。

消息队列删除函数 vQueueDelete()使用实例

```
1. #define QUEUE_LENGTH 5
2. #define QUEUE_ITEM_SIZE 4
3.
4. int main( void )
5. {
6.     QueueHandle_t xQueue;
7.     /* 创建消息队列 */
8.     xQueue = xQueueCreate( QUEUE_LENGTH, QUEUE_ITEM_SIZE );
9.
10.    if ( xQueue == NULL ) {
11.        /* 消息队列创建失败 */
12.    } else {
13.        /* 删除已创建的消息队列 */
14.        vQueueDelete( xQueue );
15.    }
16. }
```



6.复位 xQueueReset ()

队列刚被创建时，里面没有数据；使用过程中可以调用xQueueReset()把队列恢复为初始状态，此函数原型为：

```
1. /* pxQueue : 复位哪个队列；
2.  * 返回值： pdPASS(必定成功)
3.  */
4. BaseType_t xQueueReset( QueueHandle_t pxQueue);
```

7.查询

可以查询队列中有多少个数据、有多少空余空间。函数原型如下：

```
1. /*
2.  * 返回队列中可用数据的个数
3.  */
4. UBaseType_t uxQueueMessagesWaiting( const QueueHandle_t xQueue );
5.
6. /*
7.  * 返回队列中可用空间的个数
8.  */
9. UBaseType_t uxQueueSpacesAvailable( const QueueHandle_t xQueue );
```

三、消息队列使用注意事项

在使用 FreeRTOS 提供的消息队列函数的时候，需要了解以下几点：

1. 使用 xQueueSend()、xQueueSendFromISR()、xQueueReceive()等这些函数之前应先 创建需消息队列，并根据队列句柄进行操作。
2. 队列读取采用的是先进先出（FIFO）模式，会先读取先存储在队列中的数据。当然也 FreeRTOS 也支持后进先出（LIFO）模式，那么读取的时候就会读取到后进 队列的数据。
3. 在获取队列中的消息时候，我们必须定义一个存储读取数据的地方，并且该数 据区域大小不小于消息大小，否则，很可能引发地址非法的错误。
4. 无论是发送或者是接收消息都是以拷贝的方式进行，如果消息过于庞大，可以将 消息的地址作为消息进行发送、接收。

5. 队列是具有自己独立权限的内核对象，并不属于任何任务。所有任务都可以向同一队列写入和读出。一个队列由多任务或中断写入是经常的事，但由多个任务读出倒是用的比较少。

四、消息队列应用实例

消息队列实验是在 FreeRTOS 中创建了两个任务，一个是发送消息任务，一个是获取消息任务，两个任务独立运行，发送消息任务是通过检测按键的按下情况来发送消息，假如发送消息不成功，就把返回的错误代码在串口打印出来，另一个任务是获取消息任务，在消息队列没有消息之前一直等待消息，一旦获取到消息就把消息打印在串口调试助手里，

```
1. /* FreeRTOS头文件 */
2. #include "FreeRTOS.h"
3. #include "task.h"
4. #include "queue.h"
5. /* 开发板硬件bsp头文件 */
6. #include "bsp_led.h"
7. #include "bsp_usart.h"
8. #include "bsp_key.h"
9. /***** 任务句柄 *****/
10. /*
11. * 任务句柄是一个指针，用于指向一个任务，当任务创建好之后，它就具有了一个任务句柄
12. * 以后我们要想操作这个任务都需要通过这个任务句柄，如果是自身的任务操作自己，那么
13. * 这个句柄可以为NULL。
14. */
15. static TaskHandle_t AppTaskCreate_Handle = NULL; /* 创建任务句柄 */
16. static TaskHandle_t Receive_Task_Handle = NULL; /* LED任务句柄 */
17. static TaskHandle_t Send_Task_Handle = NULL; /* KEY任务句柄 */
18.
19. /***** 内核对象句柄 *****/
20. /*
21. * 信号量，消息队列，事件标志组，软件定时器这些都属于内核的对象，要想使用这些内核
22. * 对象，必须先创建，创建成功之后会返回一个相应的句柄。实际上就是一个指针，后续我
23. * 们就可以通过这个句柄操作这些内核对象。
24. *
25. * 内核对象说白了就是一种全局的数据结构，通过这些数据结构我们可以实现任务间的通信，
26. * 任务间的事件同步等各种功能。至于这些功能的实现我们是通过调用这些内核对象的函数
27. * 来完成的
28. *
29. */
30. QueueHandle_t Test_Queue = NULL;
31.
32. /***** 全局变量声明 *****/
33. /*
34. * 当我们在写应用程序的时候，可能需要用到一些全局变量。
35. */
36.
37.
38. /***** 宏定义 *****/
39. /*
40. * 当我们在写应用程序的时候，可能需要用到一些宏定义。
41. */
42. #define QUEUE_LEN 4 /* 队列的长度，最大可包含多少个消息 */
43. #define QUEUE_SIZE 4 /* 队列中每个消息大小（字节） */
44.
45. /*
46. *****/
47. * 函数声明
48. *****/
49. /*
50. static void AppTaskCreate(void); /* 用于创建任务 */
51.
52. static void Receive_Task(void* pvParameters); /* Receive_Task任务实现 */
53. static void Send_Task(void* pvParameters); /* Send_Task任务实现 */
54.
55. static void BSP_Init(void); /* 用于初始化板载相关资源 */
56.
57. /*****
58. * @brief 主函数
59. * @param 无
60. * @retval 无
61. * @note 第一步：开发板硬件初始化
62. 第二步：创建APP应用任务
63. 第三步：启动FreeRTOS，开始多任务调度
```

```

64.  *****/
65. int main(void)
66. {
67.     BaseType_t xReturn = pdPASS; /* 定义一个创建信息返回值，默认为pdPASS */
68.
69.     /* 开发板硬件初始化 */
70.     BSP_Init();
71.     printf("这是一个FreeRTOS消息队列实验！\n");
72.     printf("按下KEY1或者KEY2发送队列消息\n");
73.     printf("Receive任务接收到消息在串口回显\n\n");
74.     /* 创建AppTaskCreate任务 */
75.     xReturn = xTaskCreate((TaskFunction_t )AppTaskCreate, /* 任务入口函数 */
76.                          (const char* )"AppTaskCreate", /* 任务名字 */
77.                          (uint16_t )512, /* 任务栈大小 */
78.                          (void* )NULL, /* 任务入口函数参数 */
79.                          (UBaseType_t )1, /* 任务的优先级 */
80.                          (TaskHandle_t* )&AppTaskCreate_Handle); /* 任务控制块指针 */
81.     /* 启动任务调度 */
82.     if(pdPASS == xReturn)
83.         vTaskStartScheduler(); /* 启动任务，开启调度 */
84.     else
85.         return -1;
86.
87.     while(1); /* 正常不会执行到这里 */
88. }
89.
90.
91. /*****
92.  * @ 函数名 : AppTaskCreate
93.  * @ 功能说明: 为了方便管理，所有的任务创建函数都放在这个函数里面
94.  * @ 参数 : 无
95.  * @ 返回值 : 无
96.  *****/
97. static void AppTaskCreate(void)
98. {
99.     BaseType_t xReturn = pdPASS; /* 定义一个创建信息返回值，默认为pdPASS */
100.
101.     taskENTER_CRITICAL(); /* 进入临界区 */
102.
103.     /* 创建Test_Queue */
104.     Test_Queue = xQueueCreate((UBaseType_t )QUEUE_LEN, /* 消息队列的长度 */
105.                               (UBaseType_t )QUEUE_SIZE); /* 消息的大小 */
106.     if(NULL != Test_Queue)
107.         printf("创建Test_Queue消息队列成功!\r\n");
108.
109.     /* 创建Receive_Task任务 */
110.     xReturn = xTaskCreate((TaskFunction_t )Receive_Task, /* 任务入口函数 */
111.                          (const char* )"Receive_Task", /* 任务名字 */
112.                          (uint16_t )512, /* 任务栈大小 */
113.                          (void* )NULL, /* 任务入口函数参数 */
114.                          (UBaseType_t )2, /* 任务的优先级 */
115.                          (TaskHandle_t* )&Receive_Task_Handle); /* 任务控制块指针 */
116.     if(pdPASS == xReturn)
117.         printf("创建Receive_Task任务成功!\r\n");
118.
119.     /* 创建Send_Task任务 */
120.     xReturn = xTaskCreate((TaskFunction_t )Send_Task, /* 任务入口函数 */
121.                          (const char* )"Send_Task", /* 任务名字 */
122.                          (uint16_t )512, /* 任务栈大小 */
123.                          (void* )NULL, /* 任务入口函数参数 */
124.                          (UBaseType_t )3, /* 任务的优先级 */
125.                          (TaskHandle_t* )&Send_Task_Handle); /* 任务控制块指针 */
126.     if(pdPASS == xReturn)
127.         printf("创建Send_Task任务成功!\n\n");
128.
129.     vTaskDelete(AppTaskCreate_Handle); /* 删除AppTaskCreate任务 */
130.
131.     taskEXIT_CRITICAL(); /* 退出临界区 */
132. }
133.
134.
135.
136. /*****
137.  * @ 函数名 : Receive_Task
138.  * @ 功能说明: Receive_Task任务主体
139.  * @ 参数 :
140.  * @ 返回值 : 无
141.  *****/

```

```

142. static void Receive_Task(void* parameter)
143. {
144.     BaseType_t xReturn = pdTRUE; /* 定义一个创建信息返回值，默认为pdTRUE */
145.     uint32_t r_queue; /* 定义一个接收消息的变量 */
146.     while (1)
147.     {
148.         xReturn = xQueueReceive( Test_Queue, /* 消息队列的句柄 */
149.                                 &r_queue, /* 发送的消息内容 */
150.                                 portMAX_DELAY); /* 等待时间 一直等 */
151.         if(pdTRUE == xReturn)
152.             printf("本次接收到的数据是%d\n\n",r_queue);
153.         else
154.             printf("数据接收出错, 错误代码0x%lx\n", xReturn);
155.     }
156. }
157.
158. /*****
159.  * @ 函数名 : Send_Task
160.  * @ 功能说明: Send_Task任务主体
161.  * @ 参数 :
162.  * @ 返回值 : 无
163.  *****/
164. static void Send_Task(void* parameter)
165. {
166.     BaseType_t xReturn = pdPASS; /* 定义一个创建信息返回值，默认为pdPASS */
167.     uint32_t send_data1 = 1;
168.     uint32_t send_data2 = 2;
169.     while (1)
170.     {
171.         if( Key_Scan(KEY1_GPIO_PORT,KEY1_GPIO_PIN) == KEY_ON )
172.             /* K1 被按下 */
173.             printf("发送消息send_data1! \n");
174.             xReturn = xQueueSend( Test_Queue, /* 消息队列的句柄 */
175.                                   &send_data1, /* 发送的消息内容 */
176.                                   0 ); /* 等待时间 0 */
177.             if(pdPASS == xReturn)
178.                 printf("消息send_data1发送成功!\n\n");
179.         }
180.         if( Key_Scan(KEY2_GPIO_PORT,KEY2_GPIO_PIN) == KEY_ON )
181.             /* K2 被按下 */
182.             printf("发送消息send_data2! \n");
183.             xReturn = xQueueSend( Test_Queue, /* 消息队列的句柄 */
184.                                   &send_data2, /* 发送的消息内容 */
185.                                   0 ); /* 等待时间 0 */
186.             if(pdPASS == xReturn)
187.                 printf("消息send_data2发送成功!\n\n");
188.         }
189.         vTaskDelay(20); /* 延时20个tick */
190.     }
191. }
192.
193. /*****
194.  * @ 函数名 : BSP_Init
195.  * @ 功能说明: 板级外设初始化，所有板子上的初始化均可放在这个函数里面
196.  * @ 参数 :
197.  * @ 返回值 : 无
198.  *****/
199. static void BSP_Init(void)
200. {
201.     /*
202.     * STM32中断优先级分组为4，即4bit都用来表示抢占优先级，范围为：0~15
203.     * 优先级分组只需要分组一次即可，以后如果有其他的任务需要用到中断，
204.     * 都统一用这个优先级分组，千万不要再分组，切忌。
205.     */
206.     NVIC_PriorityGroupConfig( NVIC_PriorityGroup_4 );
207.
208.     /* LED 初始化 */
209.     LED_GPIO_Config();
210.
211.     /* 串口初始化 */
212.     USART_Config();
213.
214.     /* 按键初始化 */
215.     Key_GPIO_Config();
216. }
217.
218.
219. /*****END OF FILE*****/

```



五、实验现象

按KEY1键接收信息1，按KEY1键接收信息2.

串口配置

端口COM13 USI

波特率115200

数据位8

停止位1

校验位无

操作● 关闭串口

接收区设置

☐ ASCII ☒ HEX

☐ 停止显示 ☐ 日志模式

清空接收区 保存到文件

这是一个FreeRTOS消息队列实验！
按下KEY1或者KEY2发送队列消息
Receive任务接收到消息在串口回显

创建Test_Queue消息队列成功！
创建Receive_Task任务成功！
创建Send_Task任务成功！

发送消息send_data1！
消息send_data1发送成功！

本次接收到的数据是1

发送消息send_data2！
消息send_data2发送成功！

本次接收到的数据是2

发送消息send_data1！
消息send_data1发送成功！

本次接收到的数据是1

发送消息send_data2！
消息send_data2发送成功！

本次接收到的数据是2

CSDN @不秃也很强